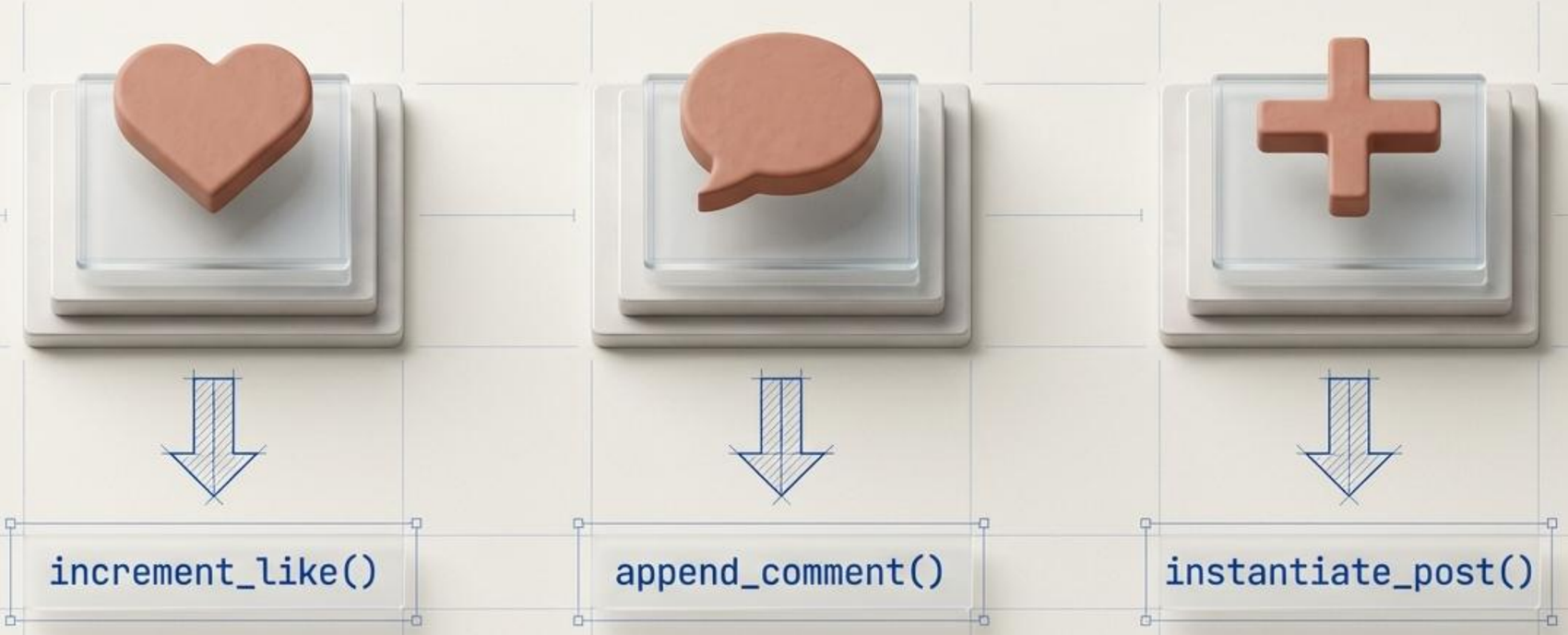
From Consumer to Creator

Building a Social Media Engine in pure Python. Applicant Experience Day.

```python
class SocialMediaFeed(object):
    def __init__(self, user):
        self.user = user
        self.posts = []

    def render_feed(self):
        feed_layout = UI.Layout.Vertical()
        feed_layout.add(UI.Header(user_profile=self.user,
                                  actions=['create']))

        post_1 = Post(author=self.user,
            image='city_arch.jpg',
            caption='Exploring the city's architecture.
                #design #urban')

        feed_layout.add(UI.PostCard(content=post_1,
            interactions=['like', 'comment', 'share'],
            comments=[
                Comment(user='Alice', text='Love this!'),
                Comment(user='Bob', text='Great shot.')])
            ]))

        post_2 = Post(author=self.user,
            image='golden_hour.jpg',
            caption='Golden hour in the park. #sunset #nature')

        feed_layout.add(UI.PostCard(content=post_2,
            interactions=['like', 'comment', 'share']))

        feed_layout.add(UI.NavigationBar(tabs=
            ['home', 'search', 'notifications', 'profile']))

    return feed_layout
```

You interact with social media every day, but today you build the engine.

We are stripping away the shiny interface, the images, and the algorithms. We will construct a basic console-based simulation using pure Python to understand how real platforms function at their core.



Deconstructing the feed into essential data.

If we strip away the graphics, what is the absolute core of a post?

Author
(Who wrote it?)

```
User = struct {
  id: UUID;
  username: String;
  profile_url: String;
}
```

Content
(What does it say?)

```
PostContent = struct {
  text_body: String;
  timestamp: DateTime;
  media_urls: List<String>;
}
```

Likes
(How popular is it?)

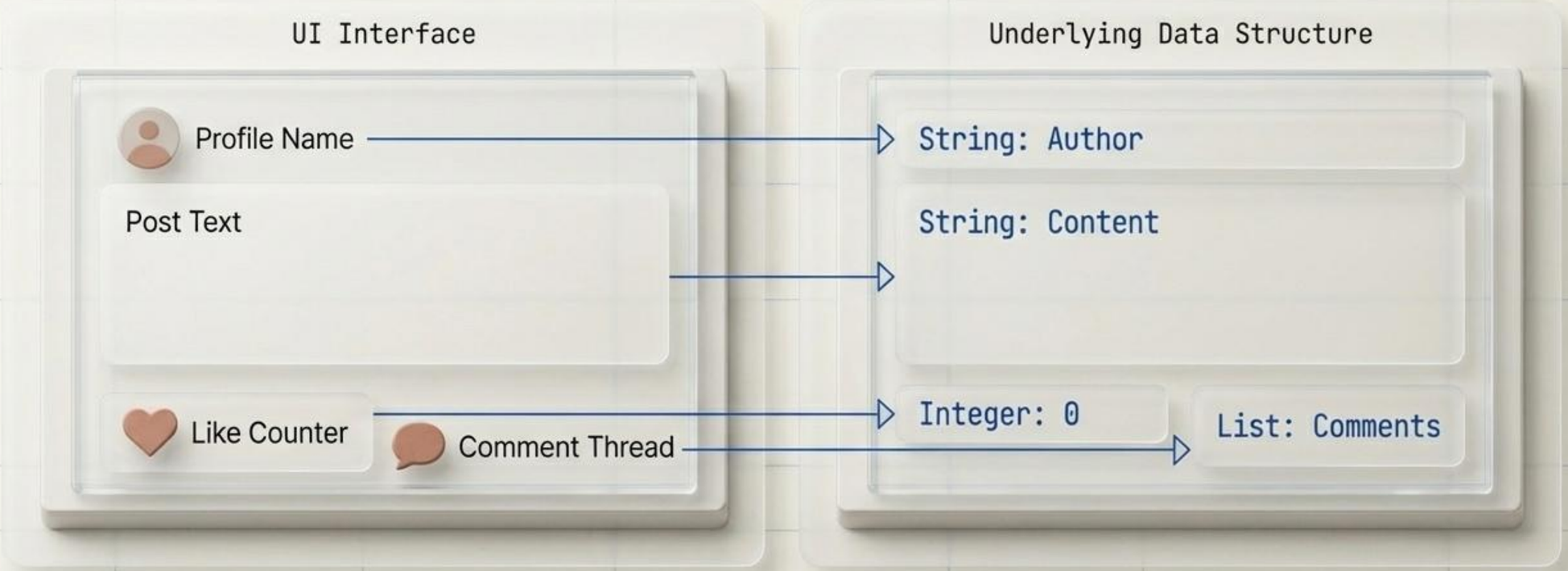
```
LikesData = struct {
  count: Integer;
  user_ids: Set<UUID>;
}
```

Comments
(What is the discussion?)

```
CommentsList =
  List<struct {
    author_id: UUID;
    text: String;
    timestamp: DateTime;
  }>
```

Translating human intuition into machine data types.

Everything you see on a screen is just a specific data structure waiting to be managed by the computer.



Core Concept: Object-Oriented Programming

Object-Oriented Programming (OOP) allows us to map real-world entities directly into code. Every post, user, and interaction is modelled as an independent ‘object’ with its own data and its own rules.

Object-Oriented Programming maps real-world interactions directly to code.

We write a single blueprint. The program uses that blueprint to spawn millions of unique posts that can be liked, shared, and commented on independently.

CLASS (The Blueprint)

```
author: <string>
likes: <integer>
content: <string>
```

OBJECTS (The Instances)

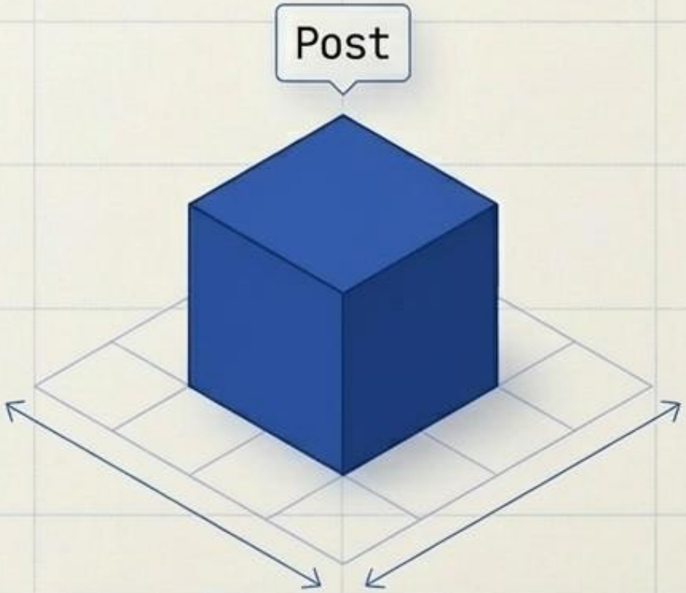
Author: Alice
Likes: 42
Content: “Just shared my new project! Check it out.”
42

Author: Bob
Likes: 0
Content: “Learning OOP principles today. Fascinating stuff.”
0

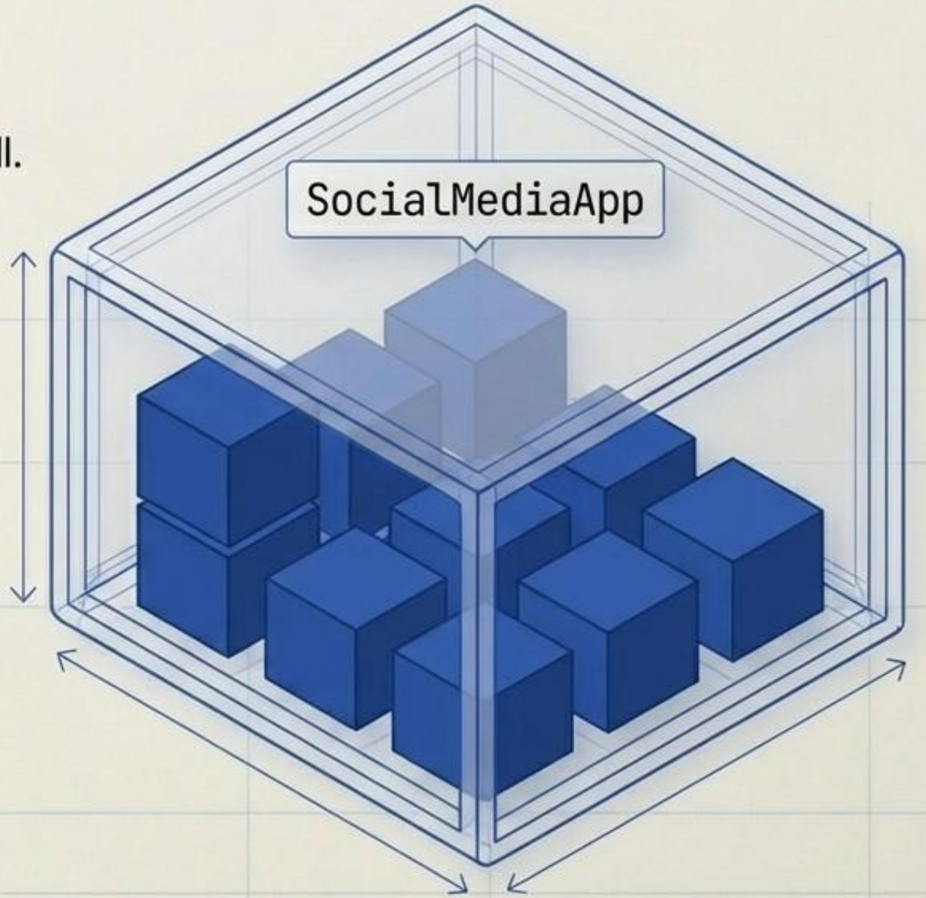
Author: Charlie
Likes: 108
Content: “Can't believe this reached 100 likes! Thanks everyone.”
108

Two core components build our entire simulation.

To model our platform, we must build a container for individual data, and a master manager to organise it all.



(The Data)



(The Manager)

Class SocialMediaApp

The overarching environment.

Manages and maintains a dynamic list of individual objects.

Class Post

The fundamental unit of content.

Represents a single piece of user-generated data moving through the system.

Distinguishing the individual data from the system manager.

Every function in our app belongs to one of these two architectural domains.

	Post		SocialMediaApp
Role	Holds and tracks individual data.	→	Manages and queries all data globally.
Properties	Stores author, content, likes, comments.	→	Stores a master list of all Posts.
Action Scope	I can be liked. / I can receive a comment.	→	I create the feed. / I find specific posts.

System Translation Matrix

Human Action	OOP Operation	Data Structure Impact
Write a status	`Instantiate Post`	Append new object to posts list
Scroll the timeline	`View Feed`	Iterate over posts list to display details
Double-tap a post	`Like Post`	Increment likes counter by 1
Reply to a thread	`Comment`	Append string to nested comments list

Defining the architecture of a single post.

We define our variables explicitly. Notice how likes and comments require specific starting states.

```
class Post:
    def __init__(self, author, content):
        self.author = author
        self.content = content
        self.likes = 0
        self.comments = []
```

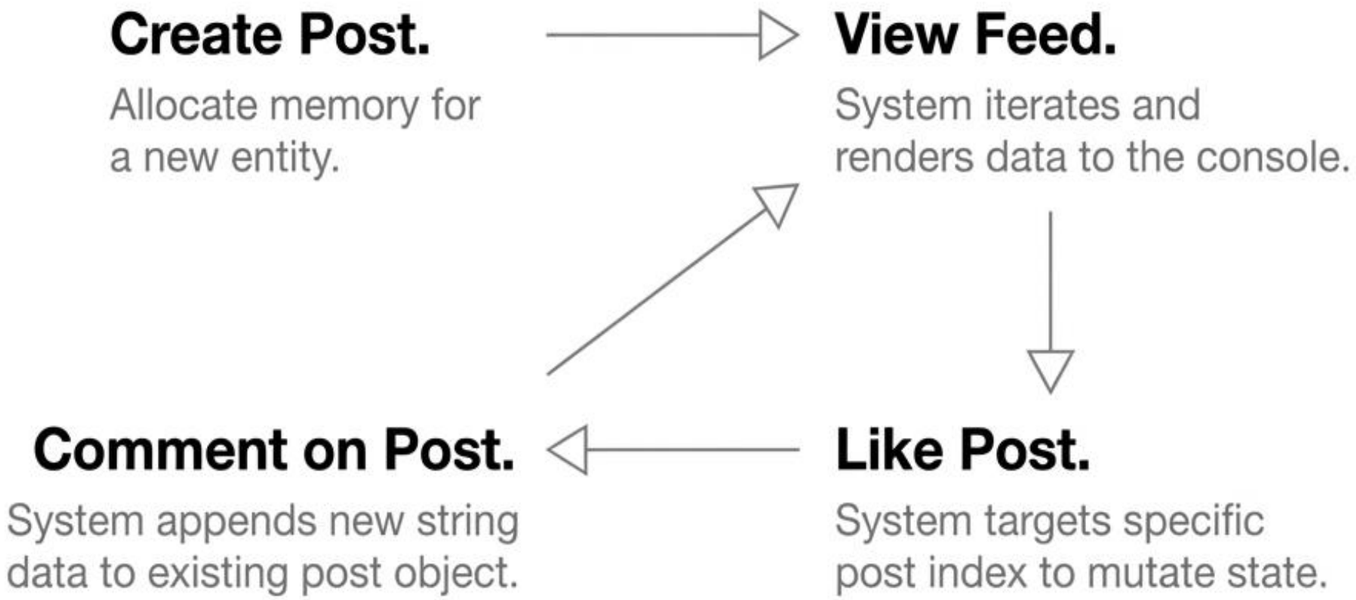
String input required.

String input required.

Every post starts at zero. Integer.

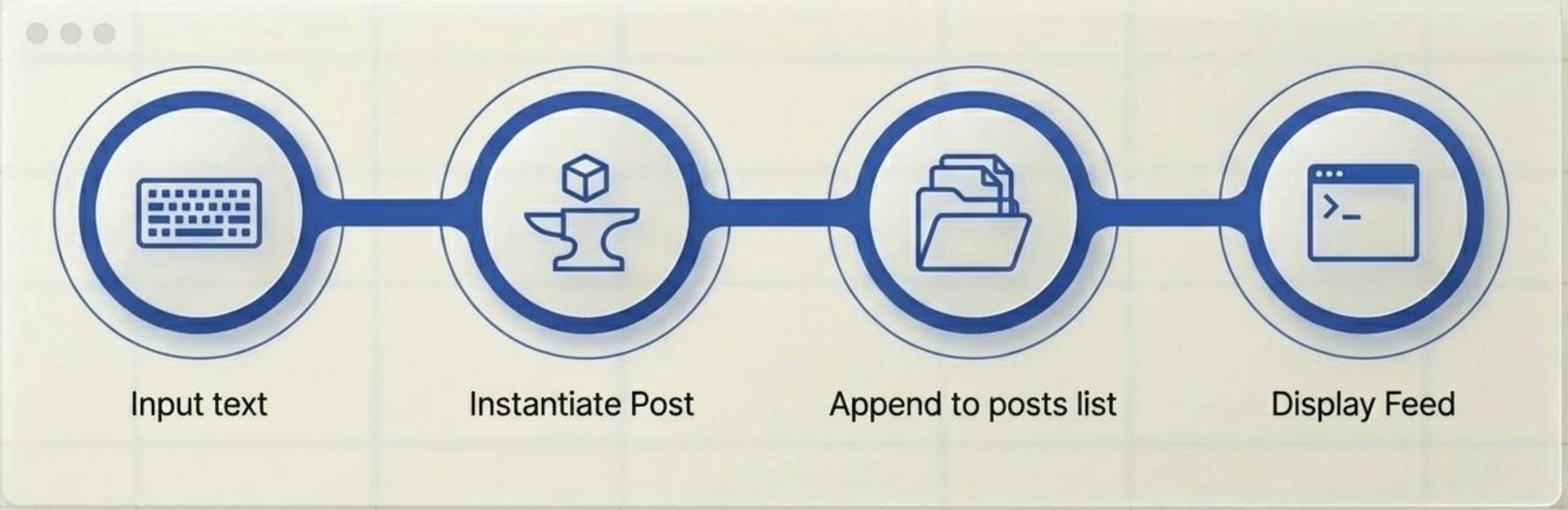
Starts as an empty List, ready to store strings.

The Lifecycle of a Post



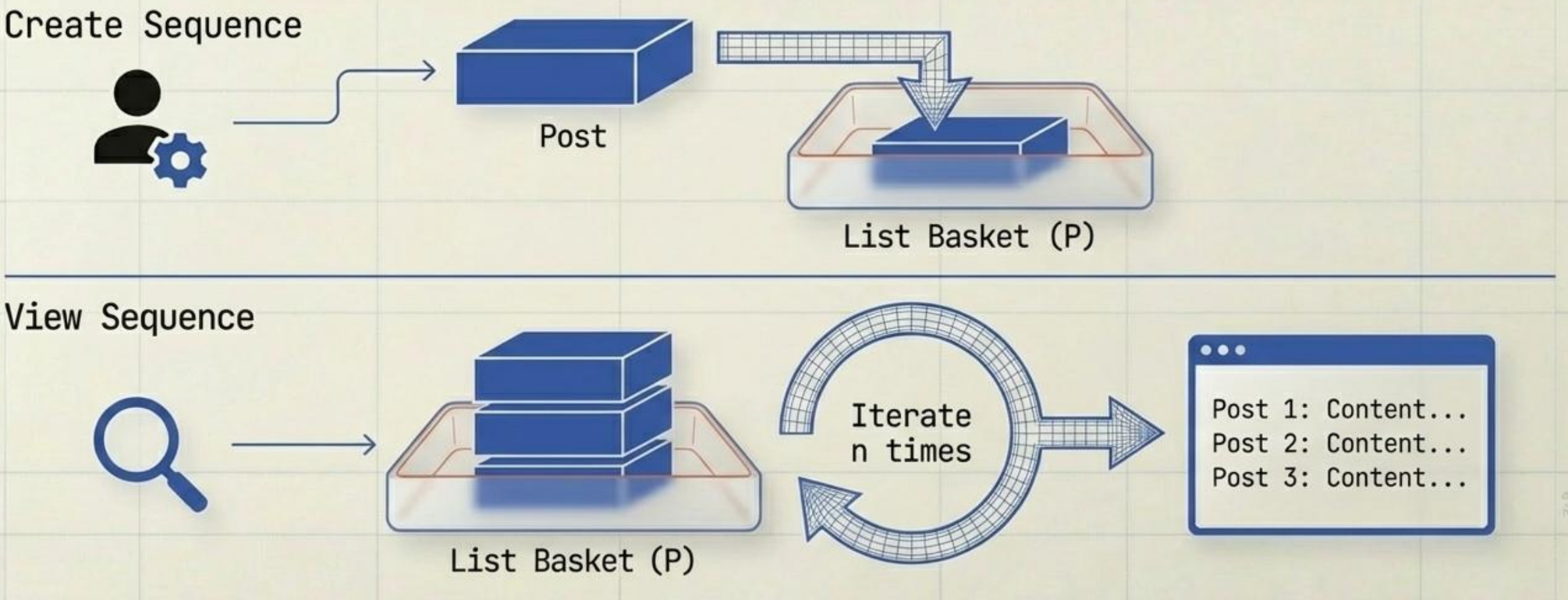
Mapping the core operations before we write the code.

We are establishing four core operations: Create Post, View Feed, Like Post, and Comment. Let's trace the algorithmic path for our primary build.



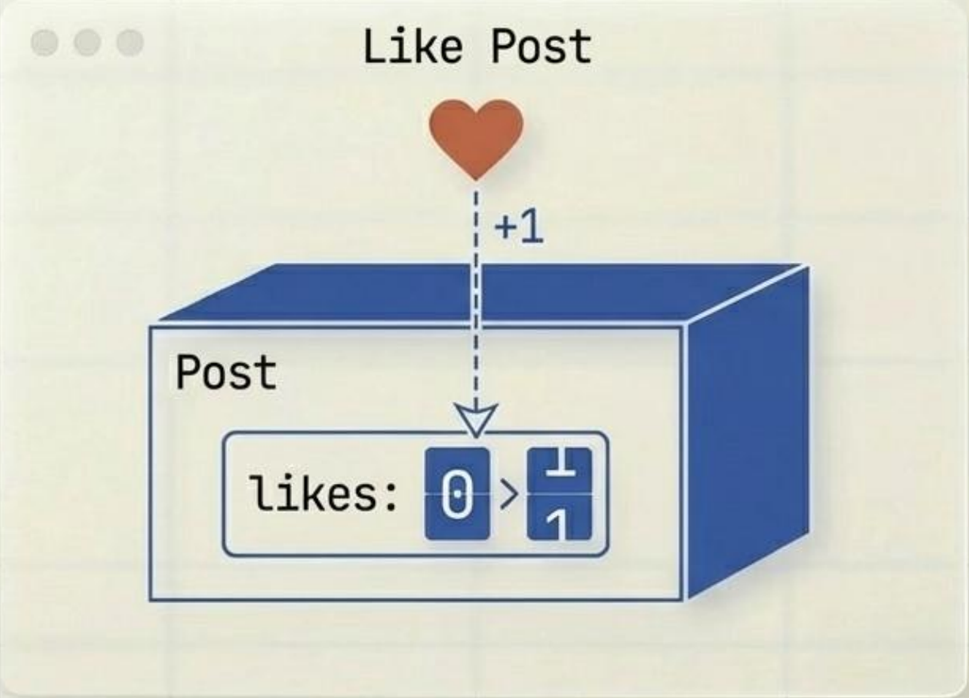
Translating thought into an algorithmic sequence.

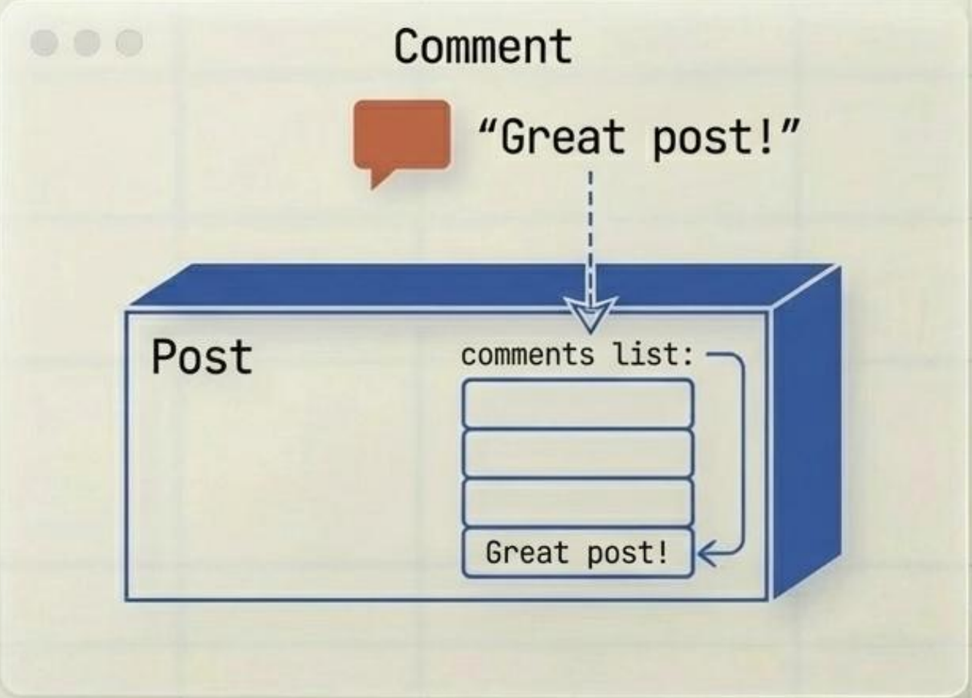
Let P represent our list of posts, and n the total number of posts.



Engagement is just incrementing counters and appending arrays.

How Object-Oriented Programming models real-world interaction within isolated objects.





The Unified Pseudocode Structure

```
Class Post:
|   Initialise author, content
|   Set likes = 0
|   Set comments = []

Class SocialMediaApp:
|   Initialise posts = []
|
|   Function Create(author, content):
|   |   New_Post = Post(author, content)
|   |   posts.Append(New_Post)
|
|   Function Like(post_index):
|   |   posts[post_index].likes += 1
```

Implement the classes and launch your feed.

Your mission: Implement the Post and SocialMediaApp classes. Create two posts, like one of them, leave a comment, and display your feed in the console.

```
--- FEED ---
[Alice]: Just setting up my console app!
Likes: 1 | Comments: ['Nice!']

[Bob]: Python makes OOP so intuitive.
Likes: 0 | Comments: []
```

Your structural skeleton for the build.

Use this framework to guide your Python script. Fill in the logical gaps.

Post Class Structure

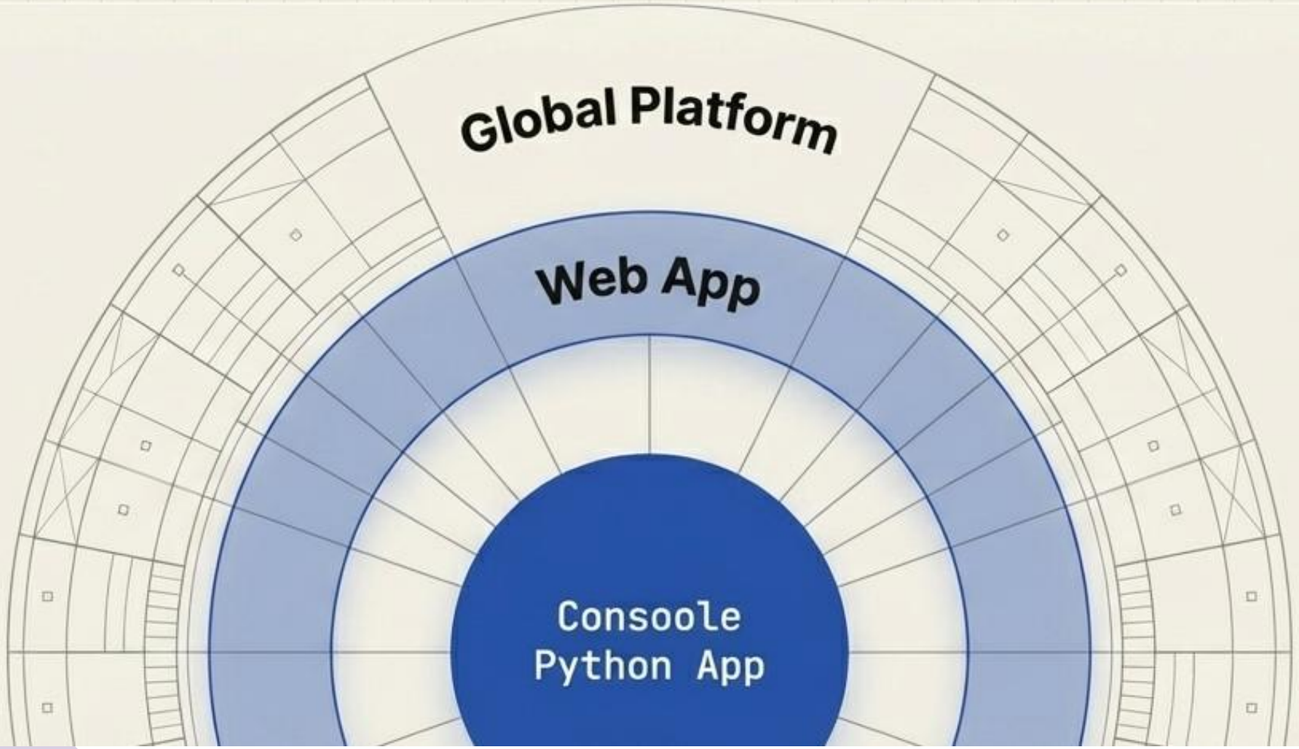
```python
class Post:
    def __init__(self, author, content):
        # TODO: initialise author
        # TODO: initialise content
        # TODO: initialise likes (0)
        # TODO: initialise comments (empty list)
```

SocialMediaApp Class Structure

```python
class SocialMediaApp:
    def __init__(self):
        self.posts = []

    def create_post(self, author, content):
        # TODO: instantiate a Post
        # TODO: append it to self.posts
```

The foundational logic you just built powers global platforms.

The architecture scales. Managing objects, incrementing variables, and storing data structures is the universal language of software engineering.



You already understand the logic.

Today, we translate that intuition into code.

Welcome to the Applicant Experience Day. Let's build.